

Experiment 11

11.1.2 Sum of digits using recursion

Algorithm :

Step 1: Start

Step 2: Input an integer n

Step 3: Call function $\text{sum}(n)$

Step 4: If $n == 0$, return 0

Step 5: Else compute:

a) $r = n \% 10$ (last digit)

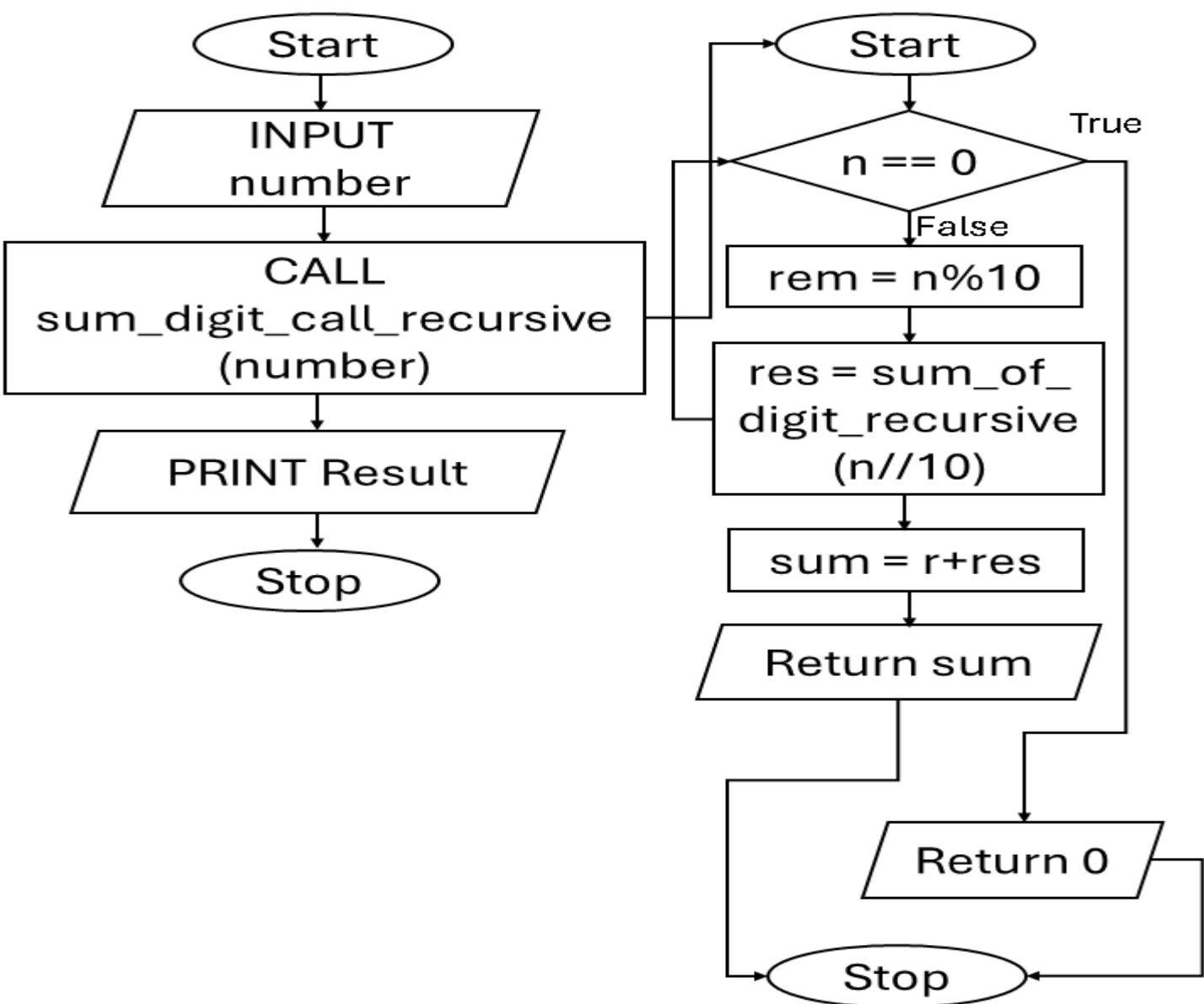
b) $q = n // 10$ (remaining number)

Step 6: Return $r + \text{sum}(q)$

Step 7: Print the result

Step 8: Stop

Flowchart :



11.1.2. Sum of Digits using Recursion

02:55

Write a recursive function `sum_of_digits_recursive(n)` that calculates the sum of the digits of a given non-negative integer n .

Input Format:

- The input consists of a single non-negative integer n .

Output Format:

- The output should be the sum of the digits of the non-negative integer n .

Constraints:

- $0 \leq n \leq 10^9$

Sample Test Cases

+

Code :

PRN : 25070521165

```
def sum_of_digits_recursive(n):  
    if n==0:  
        return 0  
    else:  
        return(n%10)+sum_of_digits_recursive(n//10)  
number=int(input())  
result = sum_of_digits_recursive(number)  
print(result)
```

Execution :

The screenshot displays a code editor with the following Python code:

```
1 def sum_of_digits_recursive(n):  
2     if n==0:  
3         return 0  
4     else:  
5         return(n%10)+sum_of_digits_recursive(n//10)  
6 number=int(input())  
7 result = sum_of_digits_recursive(number)  
8 print(result)
```

The code is being executed in a debugger. The output shows the sum of digits for a given input. The test cases are as follows:

Test Case	Input	Output	Time
Test case 1	5	5	4 ms
Test case 2	8	8	8 ms

The overall execution statistics are:

Metric	Value
Average time	0.008 s
Maximum time	0.019 s
Test cases passed	2 out of 2 shown test case(s) passed
Hidden test cases passed	3 out of 3 hidden test case(s) passed